

sdameer / My_DSA_Revision

🔍

+

<> Code

⌚ Issues

🔗 Pull requests

🛠 Agents

▶ Actions

📁 Projects

📖 Wiki

🛡 Security and quality

📈 Ins

📄

🔗 main ▾

My_DSA_Revision / Trees / main.ipynb

📄

🔍 Go to file

T

⋮

sdameer

Trees

9d6df3b · 8 hours ago

🕒

1687 lines (1687 loc) · 36.9 KB

🔗 main ▾

My_DSA_Revision / Trees / main.ipynb

📄

↑ Top

Preview

Code

Blame

📄

👤

Raw

📄

⬇

✎

▾

```
In [2]: from collections import deque, defaultdict
```

```
In [3]: class TreeNode:
        def __init__(self, val, left=None, right=None):
            self.val = val
            self.left = left
            self.right = right
```

BINARY TREE :

Example Tree

```
In [4]: root = TreeNode(1)
        root.left = TreeNode(2)
        root.right = TreeNode(3)
        root.left.left = TreeNode(4)
        root.left.right = TreeNode(5)
        root.right.left = TreeNode(6)
```

Pre - Order

In - Order

Post - Order

```
In [5]: class TreeNode:
        def __init__(self, val, left=None, right=None):
            self.val = val
            self.left = left
            self.right = right

        def preorder(node):
            if node is None:
                return

            print(node.val, end=" ")
            preorder(node.left)
            preorder(node.right)

        def inorder(node):
            if node is None:
                return

            inorder(node.left)
            print(node.val, end=" ")
            inorder(node.right)

        def postorder(node):
            if node is None:
                return

            postorder(node.left)
            postorder(node.right)
            print(node.val, end=" ")

        print("Preorder:")
        preorder(root)

        print("\nInorder:")
        inorder(root)

        print("\nPostorder:")
        postorder(root)
```

Preorder:

```

1 2 4 5 3 6
Inorder:
4 2 5 1 6 3
Postorder:
4 5 2 6 3 1

```

BFS

```

In [6]: # BFS :
def levelOrder(root):
    ans = []
    if root is None: return ans
    q = deque([root])
    while q:
        size = len(q)
        level = []
        for i in range(size):
            node = q.popleft()
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
            level.append(node.val)
        ans.append(level)
    return ans

levelOrder(root)

```

Out[6]: [[1], [2, 3], [4, 5, 6]]

DFS

```

In [7]: # DFS :
def dfs(node):
    if node is None:
        return

    print(node.val, end=" ")

    dfs(node.left)
    dfs(node.right)

root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
root.right.left = TreeNode(6)

dfs(root)

```

1 2 4 5 3 6

Pre order traversal Iterative :

Root -> Left -> Right

```

In [8]: from collections import deque

class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# preorder iterative traversal
stack = [root]
res = []
while stack:
    node = stack.pop()
    if node.right:
        stack.append(node.right)
    if node.left:

```

```

        stack.append(node.left)
        res.append(node.val)

    print(res)

```

[1, 2, 4, 5, 3, 6]

In - Order :

Left -> Root -> Right

```

In [9]: # as traversal starts with left
# so when there is a node then push to stack
# update node with left
# and when popping pop and push right to stack
stack = []
node = root
res = []

while True:
    if node:
        stack.append(node)
        node = node.left
    else: # NO node => we are at leaf node
        if not stack: # NO stack => processed whole Tree
            break

        node = stack.pop()
        res.append(node.val)
        node = node.right

print(res)

```

[4, 2, 5, 1, 6, 3]

Post - Order :

Left -> Right -> Root

```

In [10]: stack = [root]
res = []

while stack:
    node = stack.pop()
    res.append(node.val)
    if node.left :
        stack.append(node.left)
    if node.right :
        stack.append(node.right)

print(res[::-1])

```

[4, 5, 2, 6, 3, 1]

Maximum Depth in BT

```

In [11]: def height(node: TreeNode):
    if node is None:
        return 0

    left = height(node.left)
    right = height(node.right)

    return 1 + max(left, right)

height(root)

```

Out[11]: 3

Check for balanced binary tree

```

In [12]: def isBalanced(node : TreeNode):
            if node is None :
                return 0

            left = isBalanced(node.right)
            right = isBalanced(node.left)

            if abs(left - right )> 1:
                return -1

            if left == -1 : return -1
            if right == -1 : return -1
            return 1 + max(left , right)

            root = TreeNode(1)
            root.left = TreeNode(2)
            root.right = TreeNode(3)
            root.left.left = TreeNode(2)
            root.left.left.left = TreeNode(3)

            isBalanced(root) != -1

```

Out[12]: False

Diameter of Binary Tree

```

In [13]: dia = 0

def DiameterOfTree(root):
    global dia

    if root is None:
        return 0

    left = DiameterOfTree(root.left)
    right = DiameterOfTree(root.right)

    dia = max(dia, left + right)

    return max(left, right) + 1

root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)

DiameterOfTree(root)
print(dia)

```

2

Maximum path sum

```

In [14]: def maxPathSum(root):
            ans = float('-inf')

            def maxp(node):
                nonlocal ans
                if node is None : return 0
                # this code with add -ve numbers which will reduce the path
                # leftsum = maxp(node.left)
                # rightsum = maxp(node.right)
                # always remove -ve by comparing with 0
                leftsum = max(0 , maxp(node.left))
                rightsum = max(0 , maxp(node.right))
                ans = max(ans , node.val + leftsum + rightsum)
                return node.val + max(leftsum , rightsum)

            maxp(root)
            return ans

            maxPathSum(root)

```

Out[14]: 6

Check if two trees are identical or not

```
In [15]:
root_p = TreeNode(1)
root_p.left = TreeNode(2)
root_p.right = TreeNode(3)

root_q = TreeNode(1)
root_q.left = TreeNode(2)
root_q.right = TreeNode(3)

def SameTree(p : TreeNode, q : TreeNode):
    if not p and not q : # both null node !! OKAY
        return True
    if not p or not q : # a node is null
        return False
    if p.val != q.val:
        return False
    L = SameTree(p.left , q.left)
    R = SameTree(p.right , q.right)
    return L and R

SameTree(root_p, root_q)
```

Out[15]: True

Zig Zag or Spiral Traversal

```
In [16]:
def zigzag(root : TreeNode):
    if root is None:
        return []

    q = deque([root])
    res = []
    level_index = 1
    while q :
        q_len = len(q)
        level = []
        for _ in range(q_len):
            node = q.popleft()
            level.append(node.val)
            if node.left : q.append(node.left)
            if node.right : q.append(node.right)
        if level_index%2==0:
            level.reverse()
        level_index += 1
        res.append(level)

    return res

zigzag(root)
```

Out[16]: [[1], [3, 2]]

Right Side View :

```
In [17]:
def rightSideView(root):
    ans = []
    level = 0
    def dfs(node, level):
        nonlocal ans
        if node is None : return 0
        if len(ans) == level : ans.append(node.val)
        dfs(node.right , level+1)
        dfs(node.left , level+1)

    dfs(root , level)
    return ans
```

```
rightSideView(root)
# note : " level " is not being updated globally. Each recursive call gets its own copy of the level va
```



Out[17]: [1, 3]

Symmetric Tree :

```
In [18]: def isSymmetric( root):
def dfs(left , right):
    if not left and not right : return True
    if not left or not right : return False

    return (left.val == right.val and
            dfs(left.left , right.right) and
            dfs(left.right, right.left) )
return dfs(root.left,root.right)

isSymmetric(root)
```

Out[18]: False

Boundary level Traversal :

```
In [19]: def boundary(root):
leaves = []
left_nodes = []
right_nodes = []

def leaf(node):
    if node is None:
        return

    if node != root and not node.left and not node.right:
        leaves.append(node.val)

    leaf(node.left)
    leaf(node.right)

def left(node):
    if not node or (not node.left and not node.right):
        return

    left_nodes.append(node.val)

    left(node.left if node.left else node.right)

def right(node):
    if not node or (not node.left and not node.right):
        return

    right_nodes.append(node.val)

    right(node.right if node.right else node.left)

leaf(root)
left(root.left)
right(root.right)

return [root.val] + left_nodes + leaves + right_nodes[::-1]

boundary(root)
```

Out[19]: [1, 2, 3]

Vertical Traversal :

```
In [20]: ans = defaultdict(lambda: defaultdict(list))
q = deque([(root, 0, 0)])
res = []
```

```

while q:
    node, x, y = q.popleft()
    ans[x][y].append(node.val)

    if node.left:
        q.append((node.left, x-1, y + 1))
    if node.right:
        q.append((node.right, x + 1, y + 1))

for x in sorted(ans):
    temp = []
    for y in sorted(ans[x]):
        temp.extend(ans[x][y])
    res.append(temp)
print(res)

```

[[2], [1], [3]]

Top View :

In [21]:

```

ans = {}
q = deque([(root , 0)])
while q:
    node , vert = q.popleft()
    if vert not in ans:
        ans[vert] = node.val
    if node.left :
        q.append((node.left, vert-1))
    if node.right:
        q.append((node.right , vert+1))

print([ans[i] for i in sorted(ans)])

```

[2, 1, 3]

Bottom View :

In [22]:

```

ans = {}
q = deque([(root , 0)])
while q:
    node , vert = q.popleft()

    ans[vert] = node.val

    if node.left :
        q.append((node.left, vert-1))
    if node.right:
        q.append((node.right , vert+1))

print([ans[i] for i in sorted(ans)])

```

[2, 1, 3]

Root to the Node :

In [23]:

```

arr = []

def get_path(node, x):
    if not node:
        return False

    # Assume the current node may be part of the answer,
    # so add it to the path.
    arr.append(node.val)

    # Target found.
    if node.val == x:
        return True

    # Search in both subtrees.
    # If either subtree finds the target, propagate True upward.
    left = get_path(node.left, x)
    right = get_path(node.right, x)

```



```

    if left or right:
        return True

    # Reaching here means:
    # 1. Current node is not None.
    # 2. Current node itself is not the target.
    # 3. Neither the left subtree nor the right subtree contains the target.
    # Therefore, this node is not part of the root-to-target path,
    # so undo our earlier choice and remove it.
    arr.pop()

    # Tell the parent that the target was not found
    # in the subtree rooted at the current node.
    return False

get_path(root , 7)
print(arr)

```

[]

Lowest Common Ancestor :

```

In [24]: def LCA(node, p , q):
    if node is None:
        return None
    if node.val == p or node.val == q:
        return node

    left = LCA(node.left , p , q)
    right = LCA(node.right , p , q)

    if left is None : return right
    elif right is None : return left
    else :
        print(node.val)
        return node

LCA(root ,2, 3)

```

1

```
Out[24]: <__main__.TreeNode at 0x1e4229e7950>
```

Maximum Width of BT :

```

In [25]: def widthOfBinaryTree(root):
    q = deque([(root, 0)])
    ans = 0
    while q :
        start = q[0][1]
        end = q[-1][1]
        q_len = len(q)
        for _ in range(q_len):
            node , index = q.popleft()
            index = index - start
            if node.left :
                q.append((node.left , 2*index + 1))
            if node.right :
                q.append((node.right , 2*index+2))
        ans = max(ans , end - start + 1)
    return ans

widthOfBinaryTree(root)

```

```
Out[25]: 2
```

Children Sum Property in Binary Tree :

```

In [26]: def changeTree(root):
    if not root:
        return

    child_sum = 0

```

```

if root.left:
    child_sum += root.left.val
if root.right:
    child_sum += root.right.val

# Top-down adjustment
if child_sum < root.val:
    if root.left:
        root.left.val = root.val
    if root.right:
        root.right.val = root.val
else:
    root.val = child_sum

changeTree(root.left)
changeTree(root.right)

# Bottom-up correction
total = 0
if root.left:
    total += root.left.val
if root.right:
    total += root.right.val

if root.left or root.right:
    root.val = total

changeTree(root)

```

All Nodes Distance K in Binary Tree :

Minimum time taken to burn the BT from a given Node :

Amount of Time for Binary Tree to Be Infected :

In [27]:

```

def distanceK(root, target, k):
    parent = {} # parent : node
    def inorder(node):
        if not node : return
        if node.left :
            parent[node.left] = node
            inorder(node.left)
        if node.right :
            parent[node.right] = node
            inorder(node.right)

    inorder(root)
    def bfs(root, k):
        q = deque([root])
        visited = {root}
        while q :
            if k == 0:
                return [node.val for node in q]
            q_len = len(q)
            for _ in range(q_len):

                node = q.popleft()

                if node.left and node.left not in visited:
                    q.append(node.left)
                    visited.add(node.left)

                if node.right and node.right not in visited:
                    q.append(node.right)
                    visited.add(node.right)

                if node in parent and parent[node] not in visited:
                    q.append(parent[node])
                    visited.add(parent[node])

            k = k - 1

        return []

    return bfs(target , k)

```

Count total nodes in a complete BT :

In [28]:

```
def countNodes(root) :
    def findleft(node):
        h = 0
        while (node):
            h += 1
            node = node.left
        return h

    def findright(node):
        h = 0
        while node :
            h += 1
            node = node.right
        return node

    if not root :
        return 0
    left = findleft(root.left)
    right = findright(root.right)
    if left == right :
        return 2^left - 1

    return 1 + countNodes(root.left) + countNodes(root.right)

countNodes(root)
```

Out[28]: 3

Construct Binary Tree from InOrder and PreOrder

In [29]:

```
def buildTree(preorder,inorder):
    if not preorder and not inorder : return None
    _index = {}
    for i , j in enumerate(inorder):
        _index[j] = i

    root = TreeNode(preorder[0])
    mid = _index.get(preorder[0])
    root.left = buildTree(preorder[1:mid+1], inorder[:mid])
    root.right = buildTree(preorder[mid+1:] , inorder[mid+1:])
    return root

buildTree(
    [3, 9 , 15 , 20 , 7],
    [9 , 3, 20 , 15, 7]
).val
```

Out[29]: 3

Construct Binary Tree from InOrder and PostOrder

In [30]:

```
def buildTree(inorder, postorder):
    _index = {}
    for i, j in enumerate(inorder):
        _index[j] = i

    def helper(l, r):
        if l > r:
            return None
        root = TreeNode(postorder.pop())
        idx = _index.get(root.val)
        root.right = helper(idx+1, r)
        root.left = helper(l, idx-1)
        return root

    return helper(0, len(inorder) - 1)

buildTree(
    [9, 3, 20, 15, 7],
    [9, 15, 7, 20, 3])
```

```

    ).val

```

Out[30]: 3

```

In [31]: root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
root.right.left = TreeNode(6)

```

Serialize and Deserialize Binary Tree :

```

In [32]: def serializer(root):
    if not root :
        return ""
    q = deque([root])
    ans = []
    while q :
        node = q.popleft()
        if node :
            ans.append(str(node.val))
            q.append(node.left)
            q.append(node.right)
        else :
            ans.append("#")

    return ",".join(ans)

serializer(root)

```

Out[32]: '1,2,3,4,5,6,#,#,#,#,#,#'

```

In [33]: def deserialize(data):
    values = data.split(",")
    root = TreeNode(int(values[0]))
    q = deque([root])
    i = 1
    while q :
        node = q.popleft()
        if values[i]!="#":
            node.left = TreeNode(int(values[i]))
            q.append(node.left)
        i = i + 1
        if values[i] != "#":
            node.right = TreeNode(int(values[i]))
            q.append(node.right)
        i = i + 1

    return root

deserialize('1,2,3,4,5,6,#,#,#,#,#,#').val

```

Out[33]: 1

Morris Traversal :

```

In [34]: def FindPredecessor(node):
    temp = node.left
    while temp.right and temp.right != node:
        temp = temp.right
    return temp

ans = []
def morris(node):
    curr = node
    while curr!= None :
        if not curr.left :
            ans.append(curr.val)
            curr = curr.right
        else :

```

```

pre = FindPredecessor(curr)
if not pre.right :
    pre.right = curr
    curr = curr.left
else :
    pre.right = None
    ans.append(curr.val)
    curr = curr.right

return ans

morris(root)

```

Out[34]: [4, 2, 5, 1, 6, 3]

Flatten Binary Tree to Linked List :

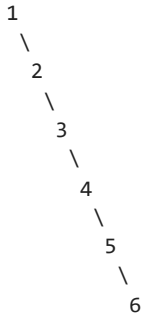
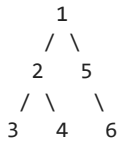
```

In [35]: def FindPredecessor(node):
temp = node.left
while temp.right:
    temp = temp.right
return temp

def flatten(node):
curr = node
while curr!= None :
    if curr.left:
        pre = FindPredecessor(curr)

        pre.right = curr.right
        curr.right = curr.left
        curr.left = None
    curr = curr.left

```



BINARY SEARCH TREE :

Search in BST :

```

In [41]: def searchBST(root, val):
while root is not None and root.val != val:
    root = root.left if val < root.val else root.right
return root

searchBST(root , val=1).val

```

Out[41]: 1

Ceil in BST :

```
In [ ]: def findceil(root, key):
        ceil = -1
        while root :
            if root.val == key:
                ceil = root.val
                return ceil

            # val >= key ✓
            if root.val > key:
                ceil = root.val
                root = root.left
            else:
                root = root.right

        return ceil

        findceil(root , key=2)
```

Out[]: 6

Floor in BST :

```
In [46]: def findfloor(root , key):
        floor = -1
        while root :
            if root.val == key :
                floor = root.val
                return floor

            # val <= key ✓
            if root.val < key:
                floor = root.val
                root = root.right
            else :
                root = root.left
        return root

        findceil(root , key=2)
```